CPSC 313: Computer Hardware and Operating Systems

Unit 2: Pipelining

Pipelining: Forwarding

Admin

- Quiz 2:
  - Next week, please ensure you've reserved your slot.
- Labs
  - Lab 4 is due on Sunday

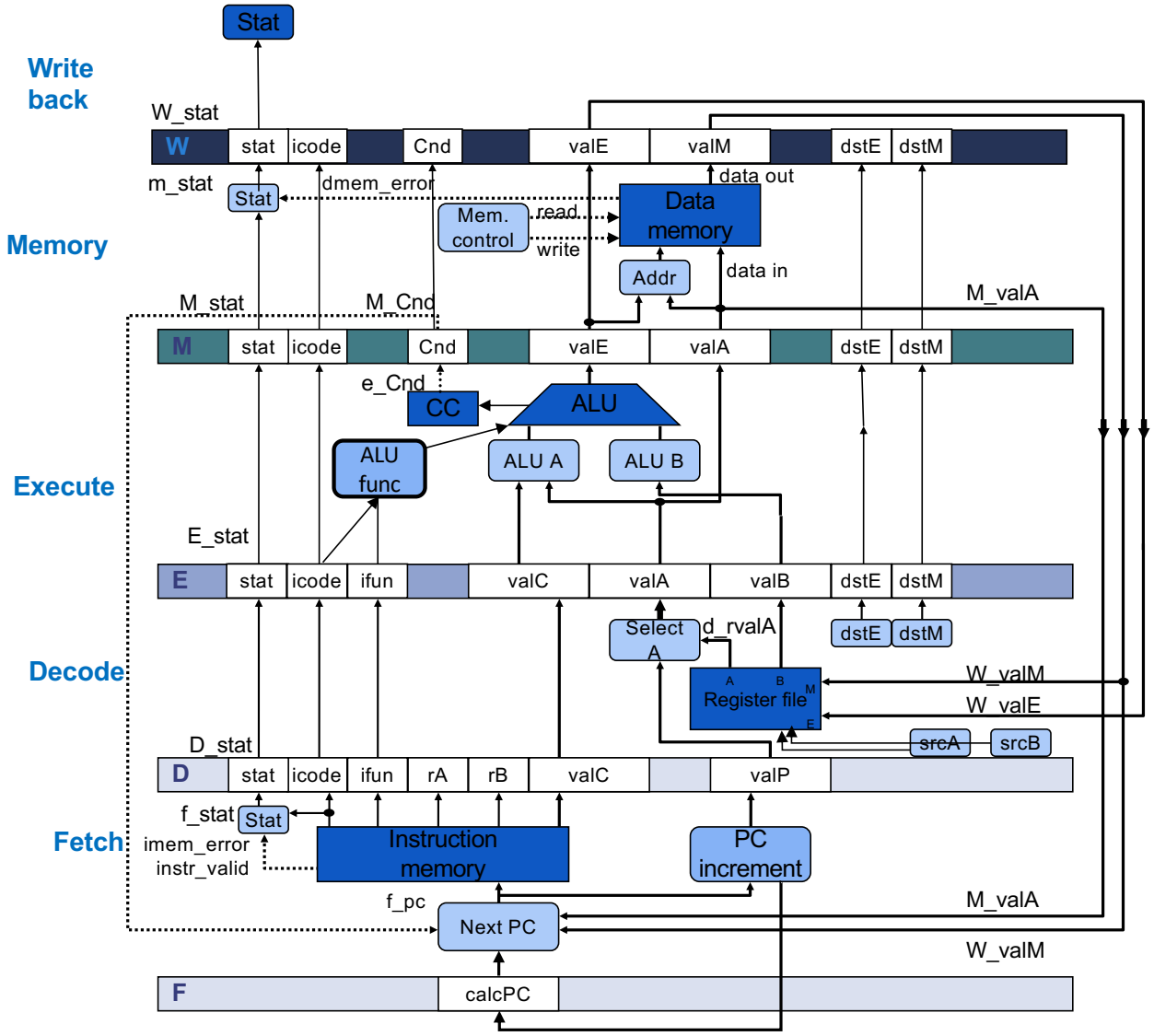
Today

- Learning outcomes
  - Identify places in the implementation where we can forward signals to avoid stalling the processor.

Recall

Consider the following sequence:

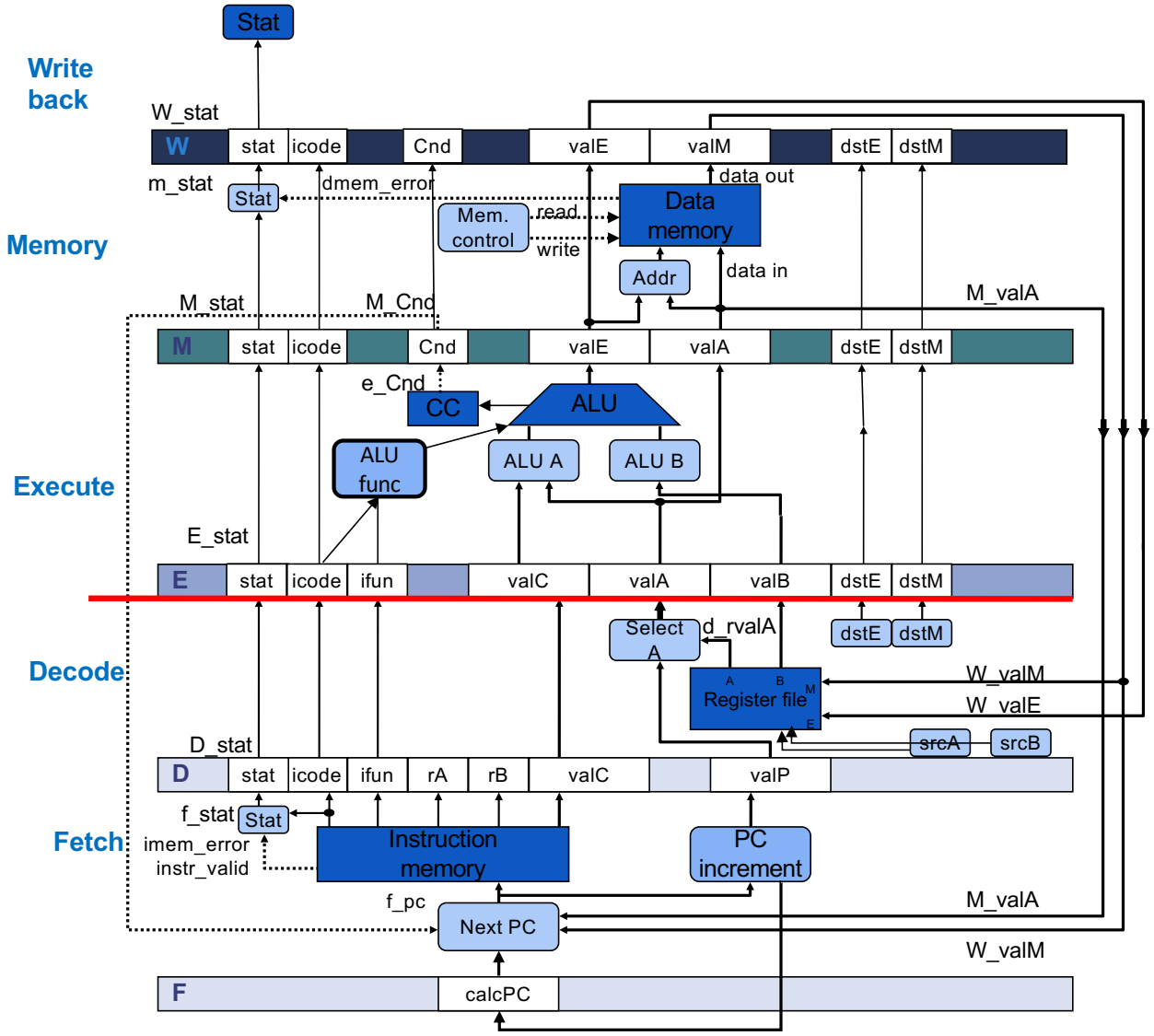
addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000



Recall

Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000



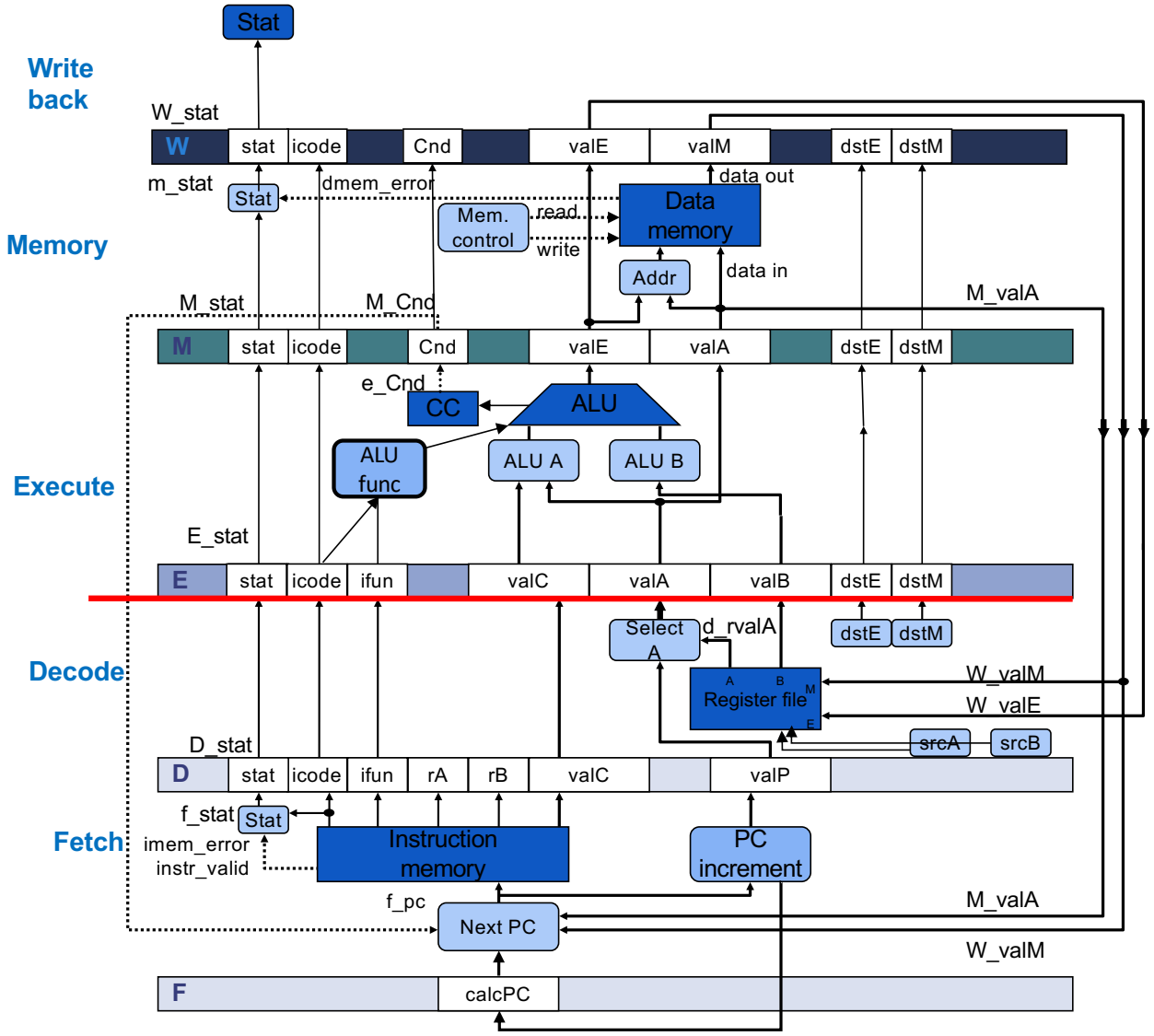
Recall stalling

Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

Until %rbx is written back:

- F and D pipeline registers maintain
- their values.
- E inserts bubbles.



Recall stalling

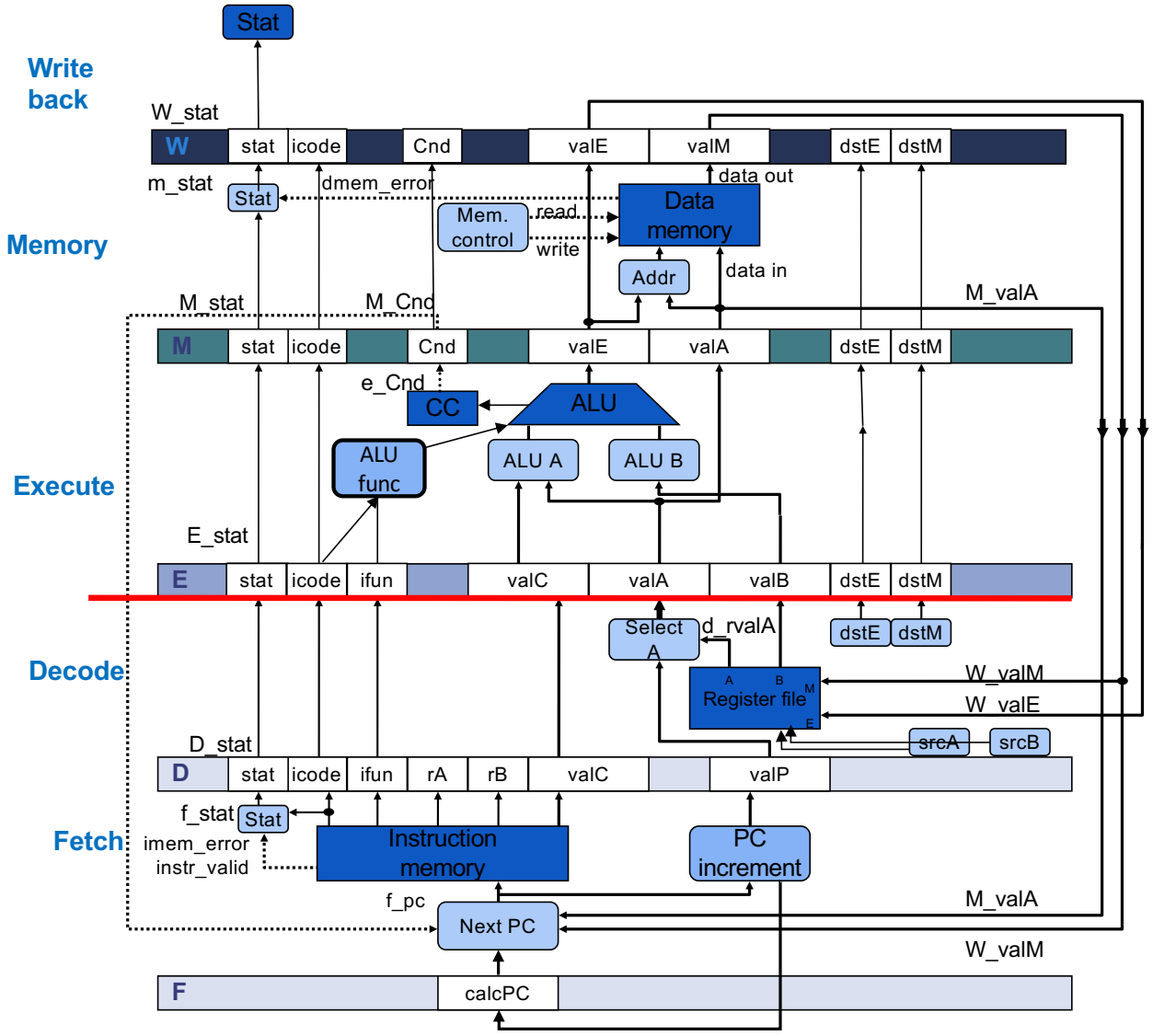
Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

Until %rbx is written back:

- F and D pipeline registers maintain
- their values.
- E inserts bubbles.

addq	F	D	E	M	W		
subq		F	D	D	D	D	E
jmp			F	F	F	F	D

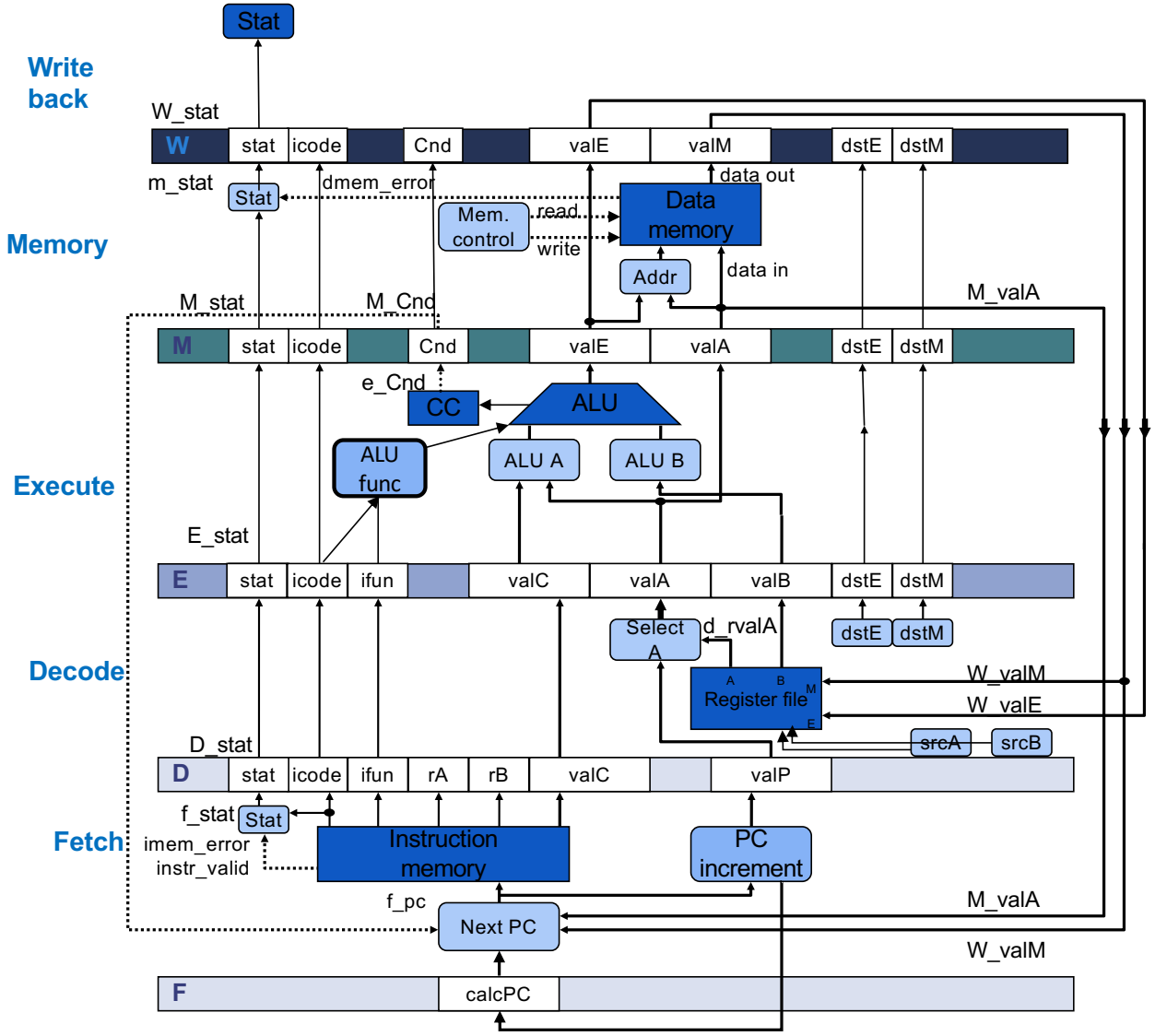


A Fix?

Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

By what point does subq need the value of %rbx?

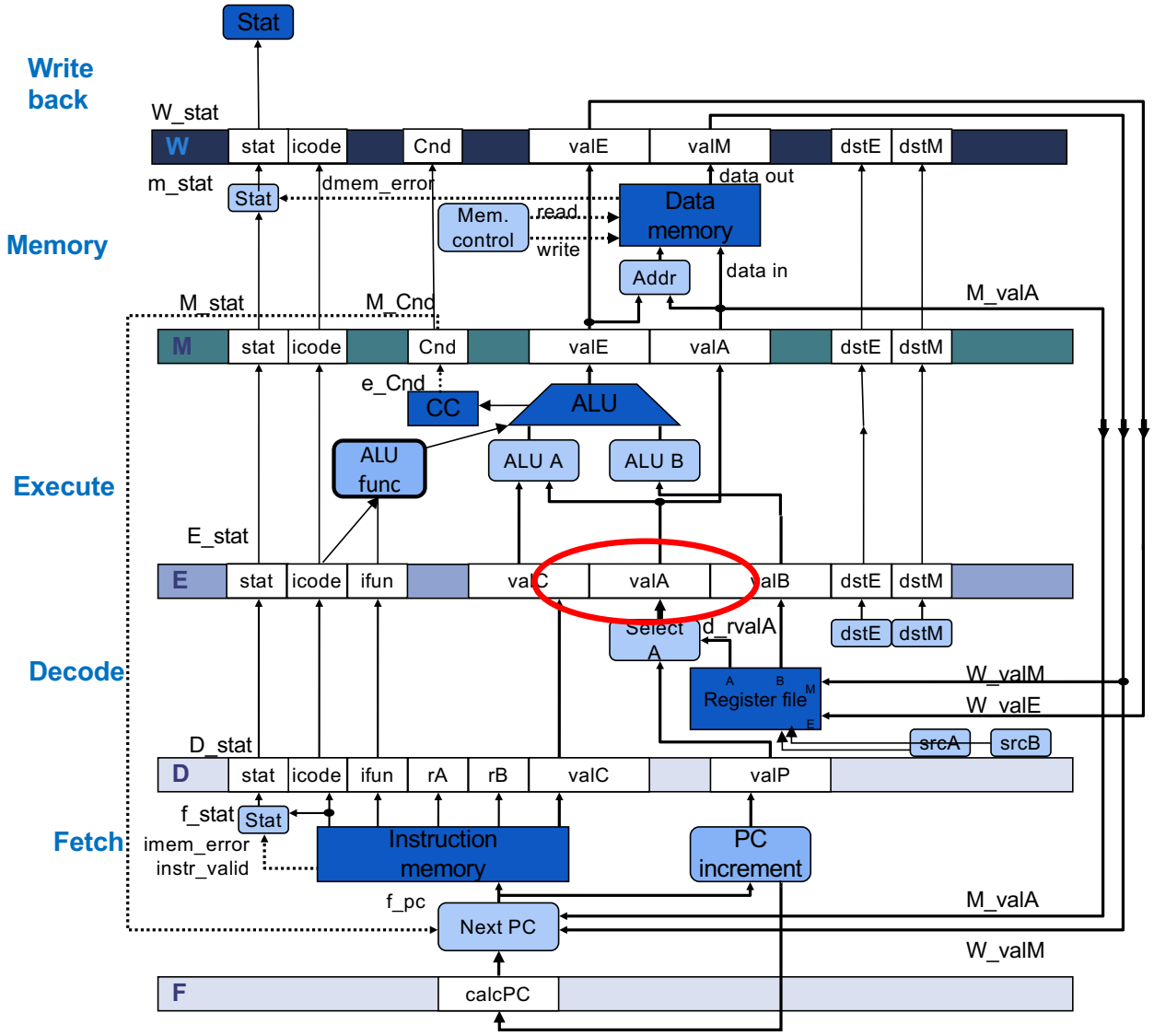


A Fix?

Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

By what point does subq need the value of %rbx?



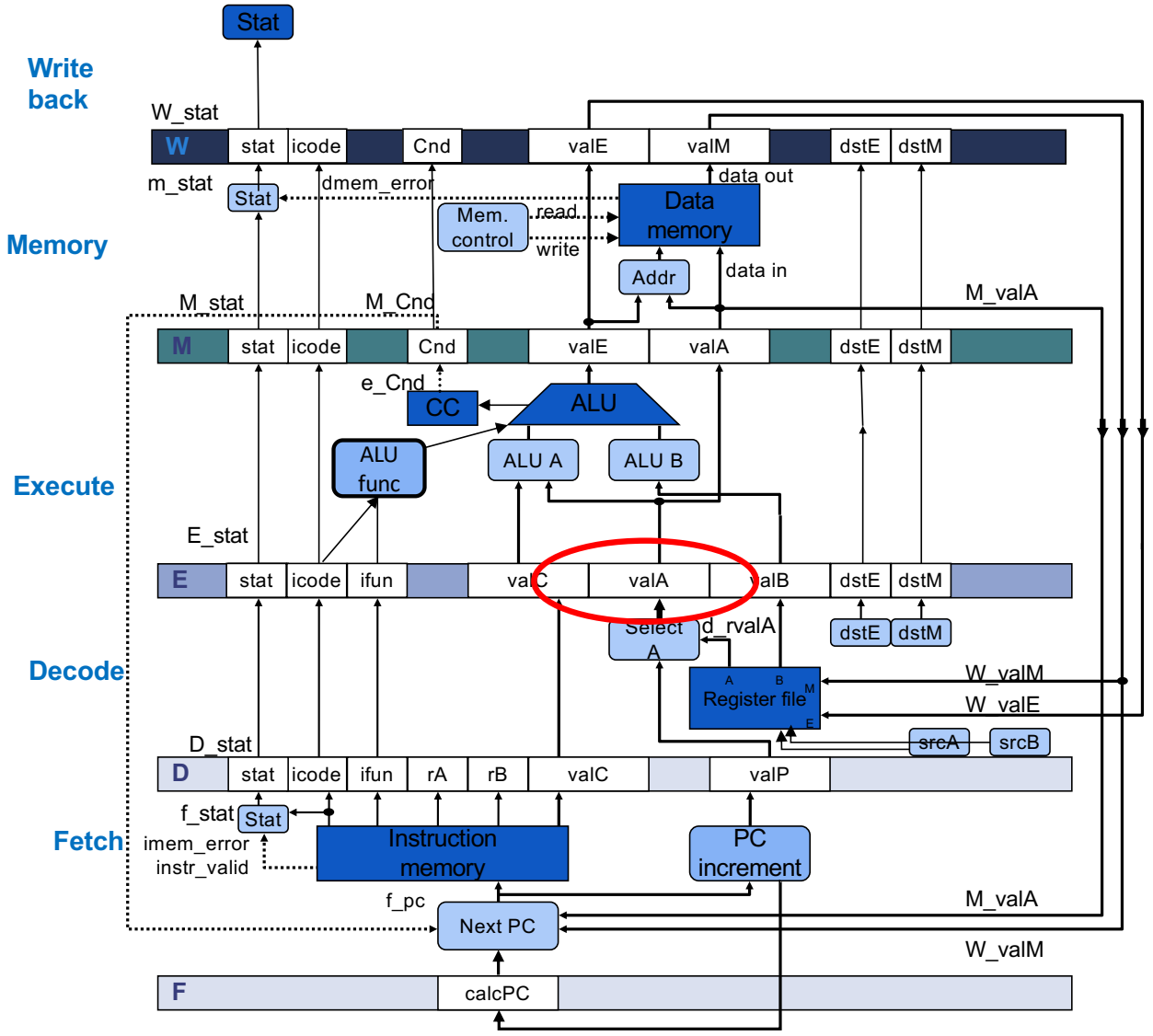
A Fix?

Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

By what point does subq need the value of %rbx?

At what point does addq have the value of %rbx?



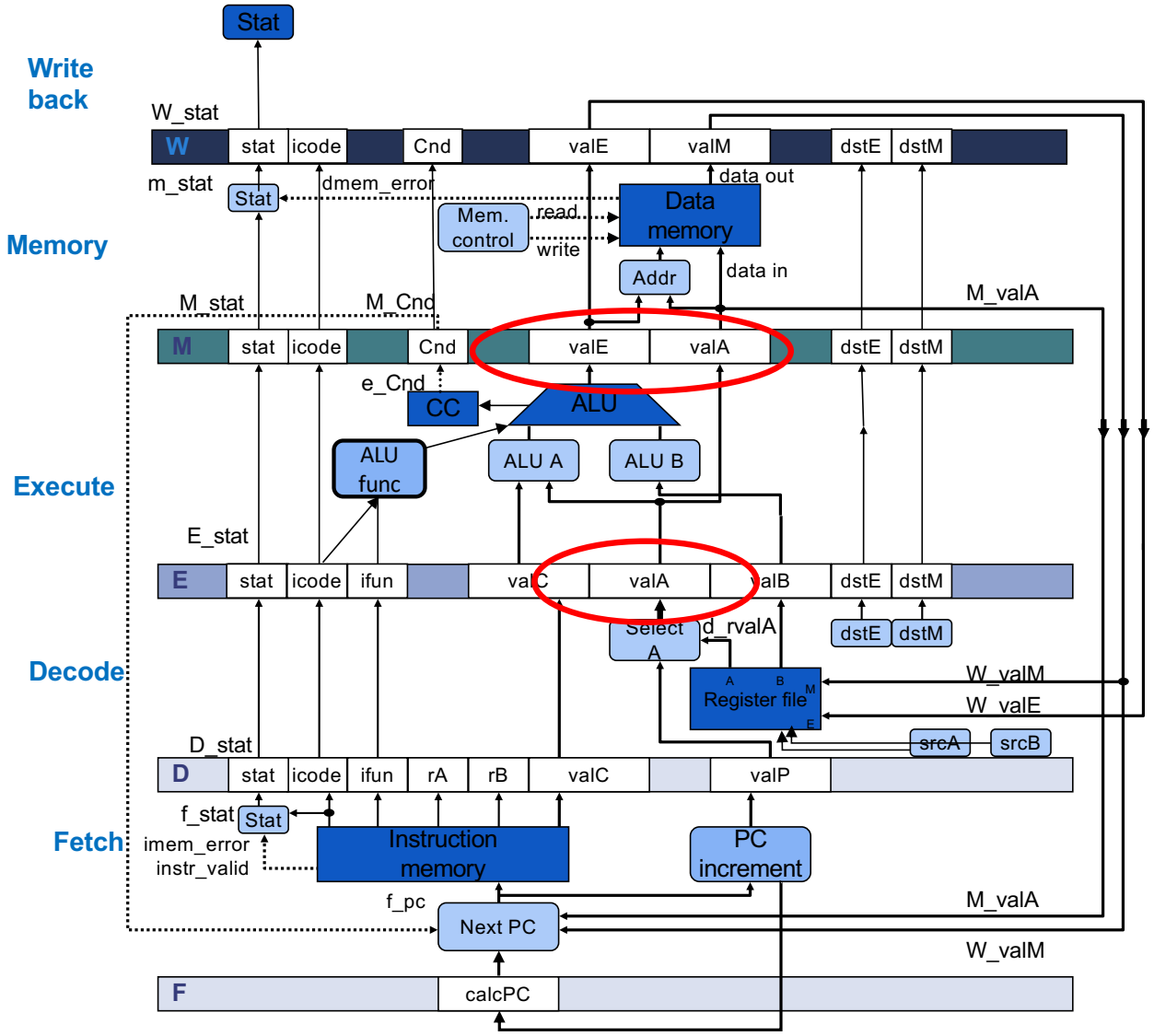
A Fix?

Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

By what point does subq need the value of %rbx?

At what point does addq have the value of %rbx?



Forwarding (1)

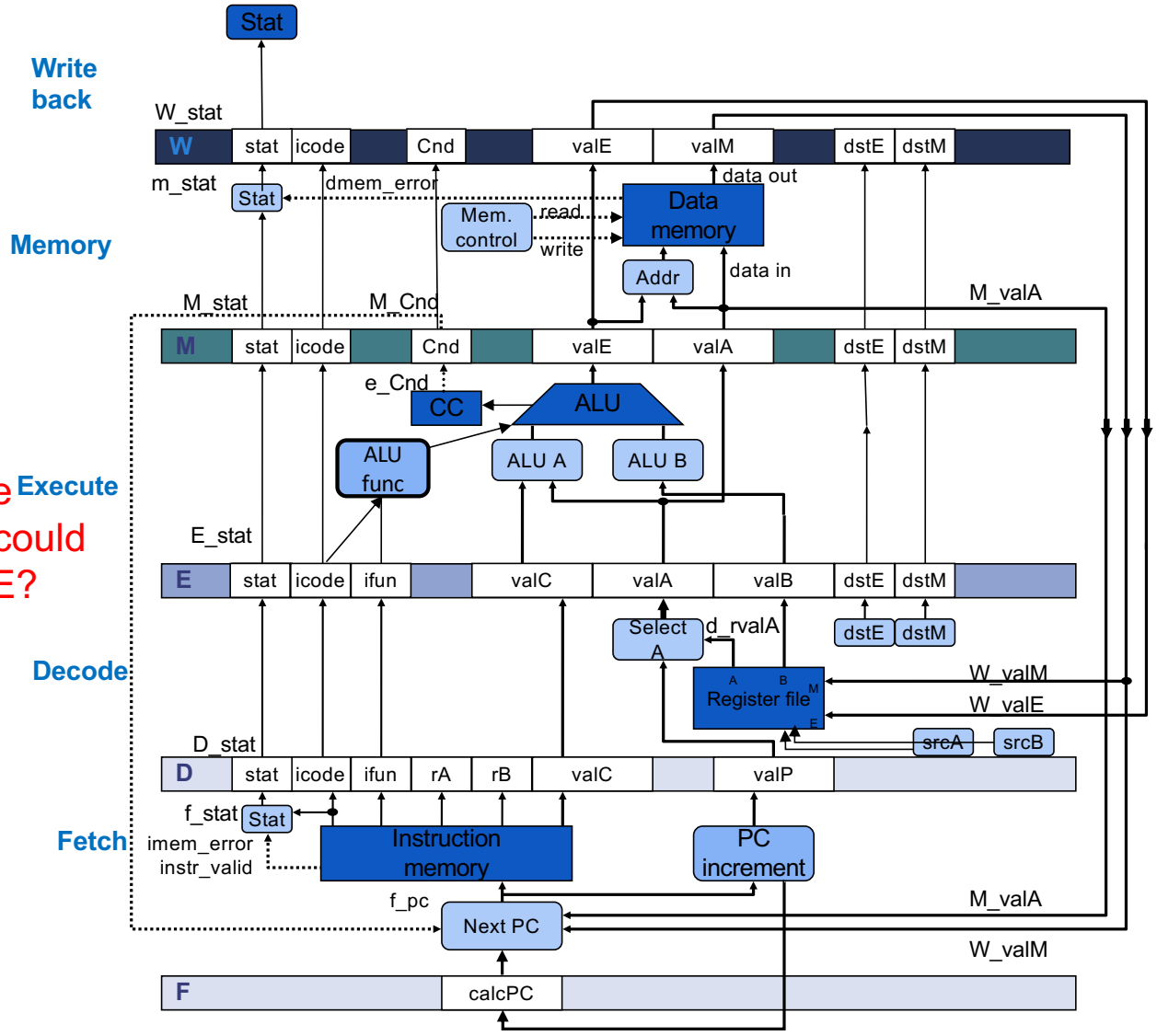
Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

We **have** the value we need before it's written into the register file, so could we somehow use the signal e_valE?

Q1: When do we NEED the value for subq?

addq	F	D	E	M	W		
subq		F	D	D	D	D	E
jmp			F	F	F	F	D



Forwarding (2)

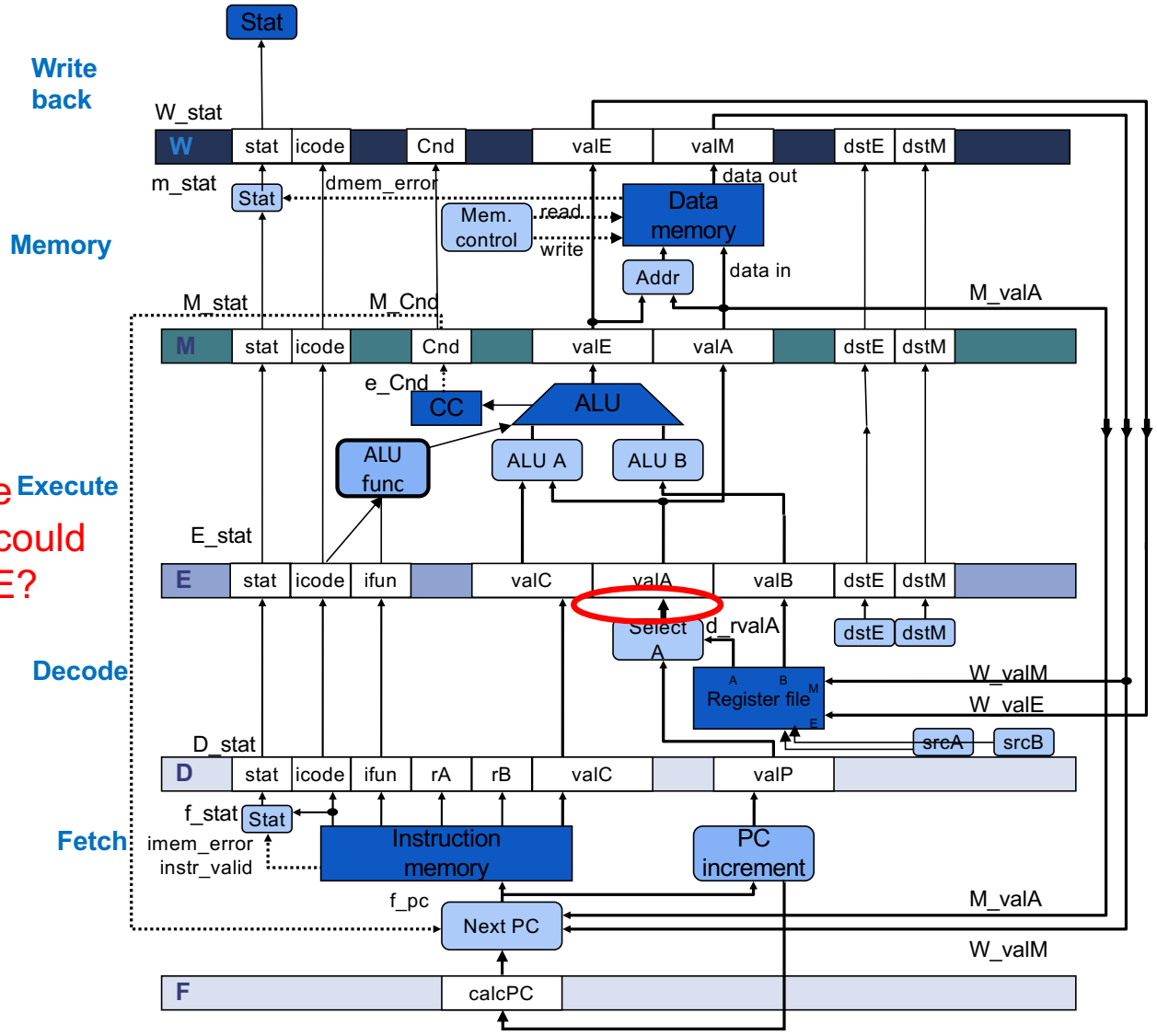
Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

We **have** the value we need before it's written into the register file, so could we somehow use the signal e_valE?

Q1: When do we NEED the value for subq?

addq	F	D	E	M	W		
subq		F	D	D	D	D	E
jmp			F	F	F	F	D



Forwarding (2)

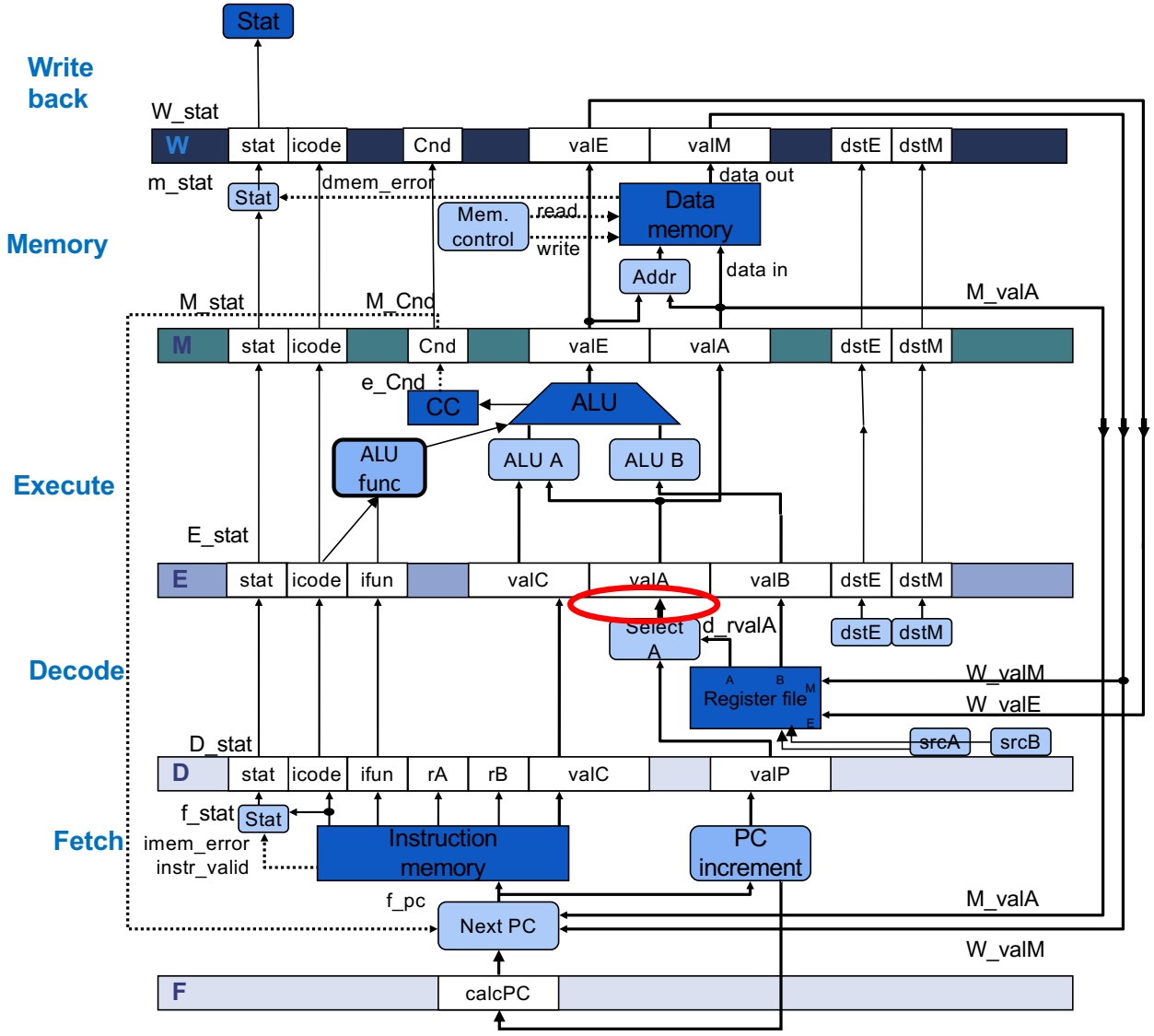
Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

Q1: When do we NEED the value for subq?

Q2: When do we HAVE the value from addq?

addq	F	D	E	M	W		
subq		F	D	D	D	D	E
jmp			F	F	F	F	D



Forwarding (2)

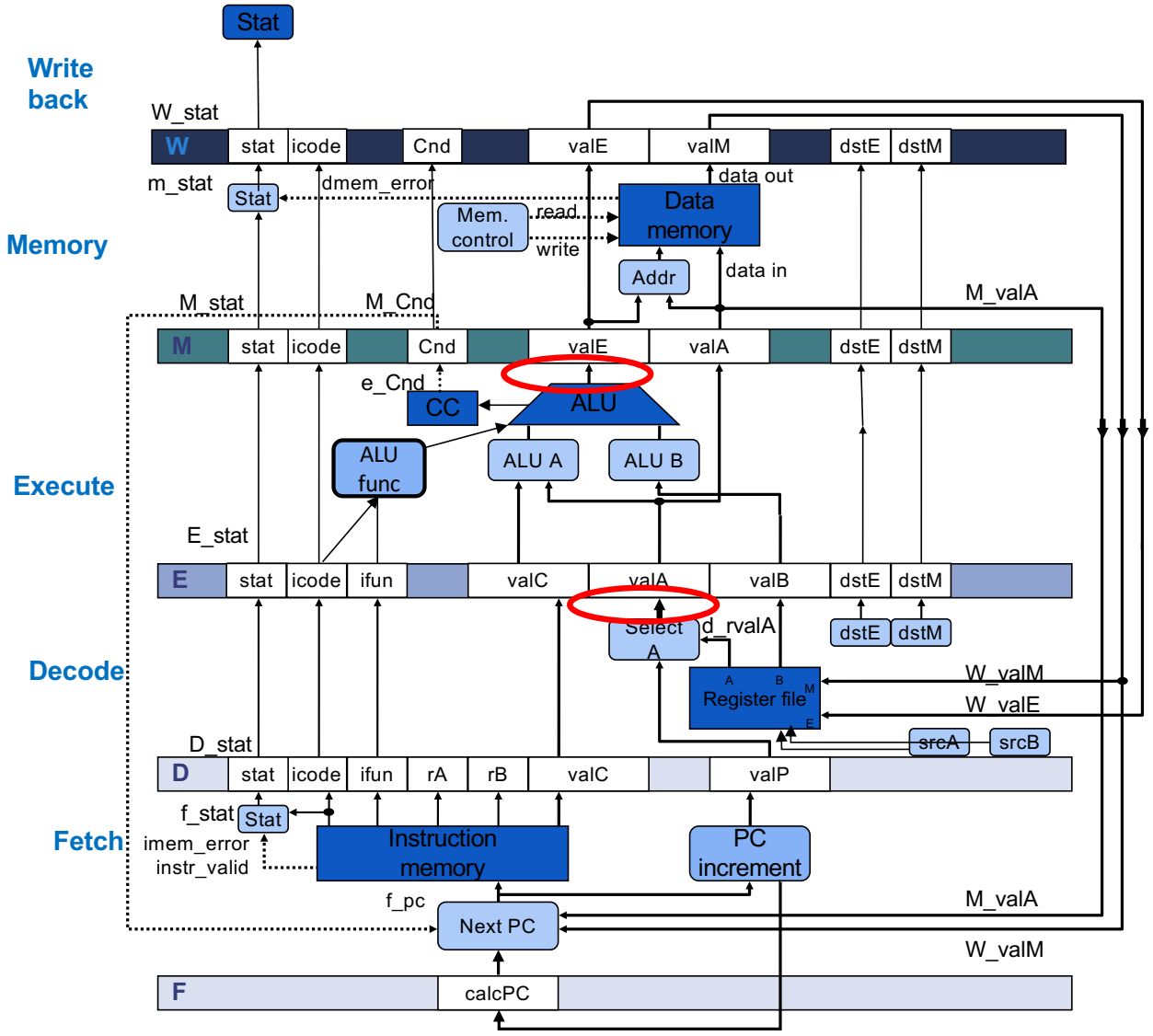
Consider the following sequence:

addq %rax, %rbx
subq %rbx, %rcx
jmp 0x2000

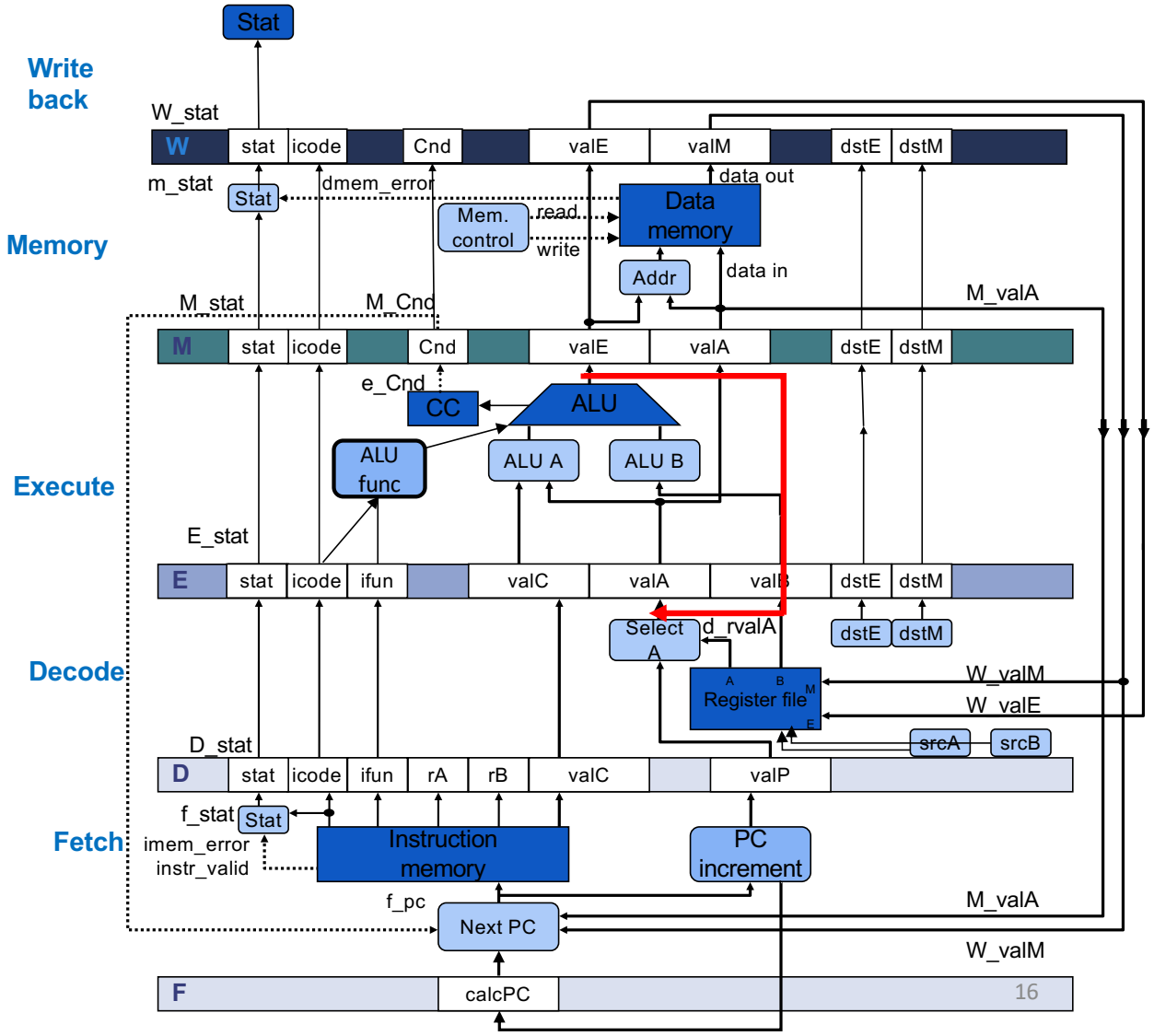
Q1: When do we NEED the value for subq?

Q2: When do we HAVE the value from addq?

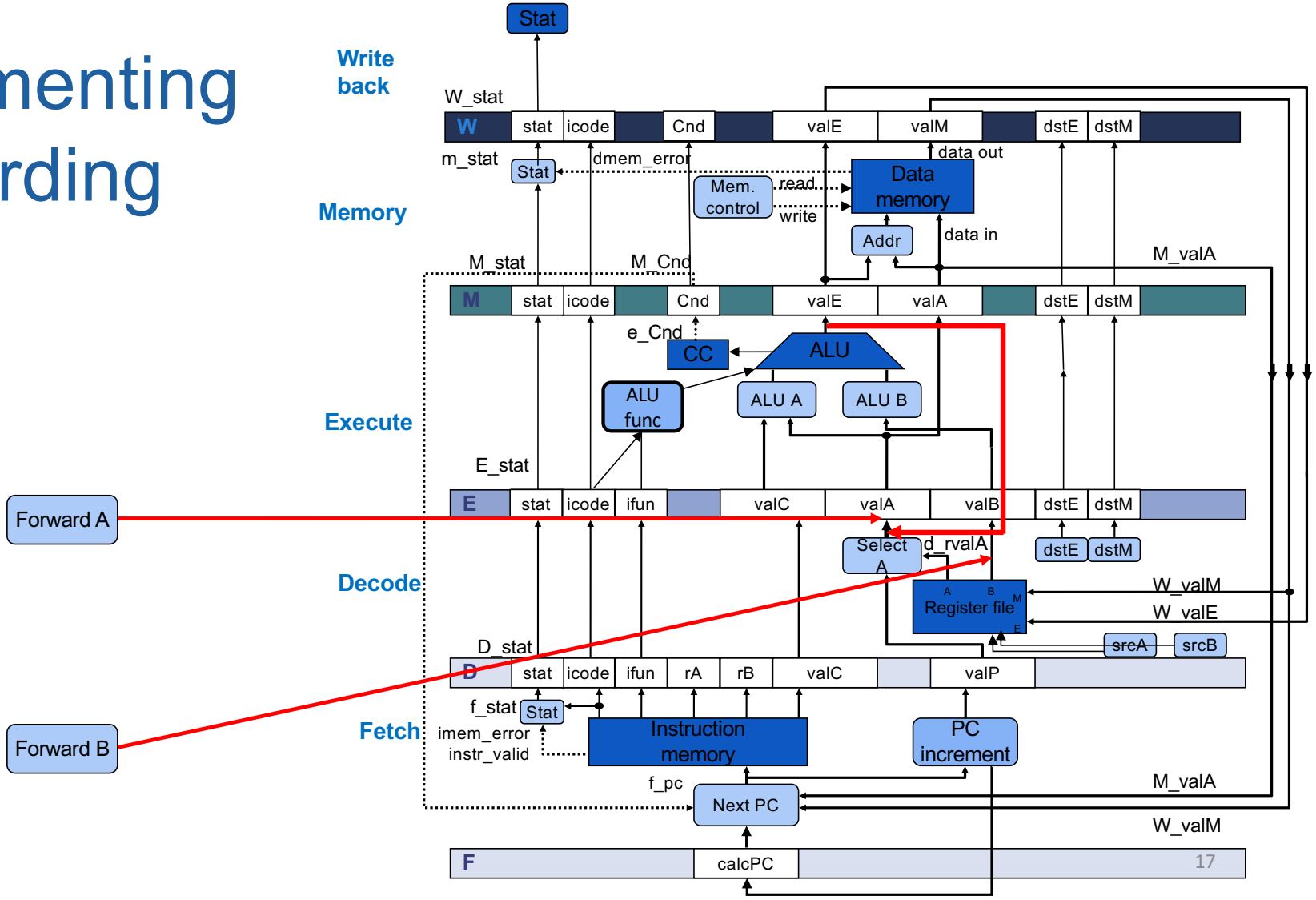
addq	F	D	E	M	W		
subq		F	D	D	D	D	E
jmp			F	F	F	F	D



Implementing Forwarding

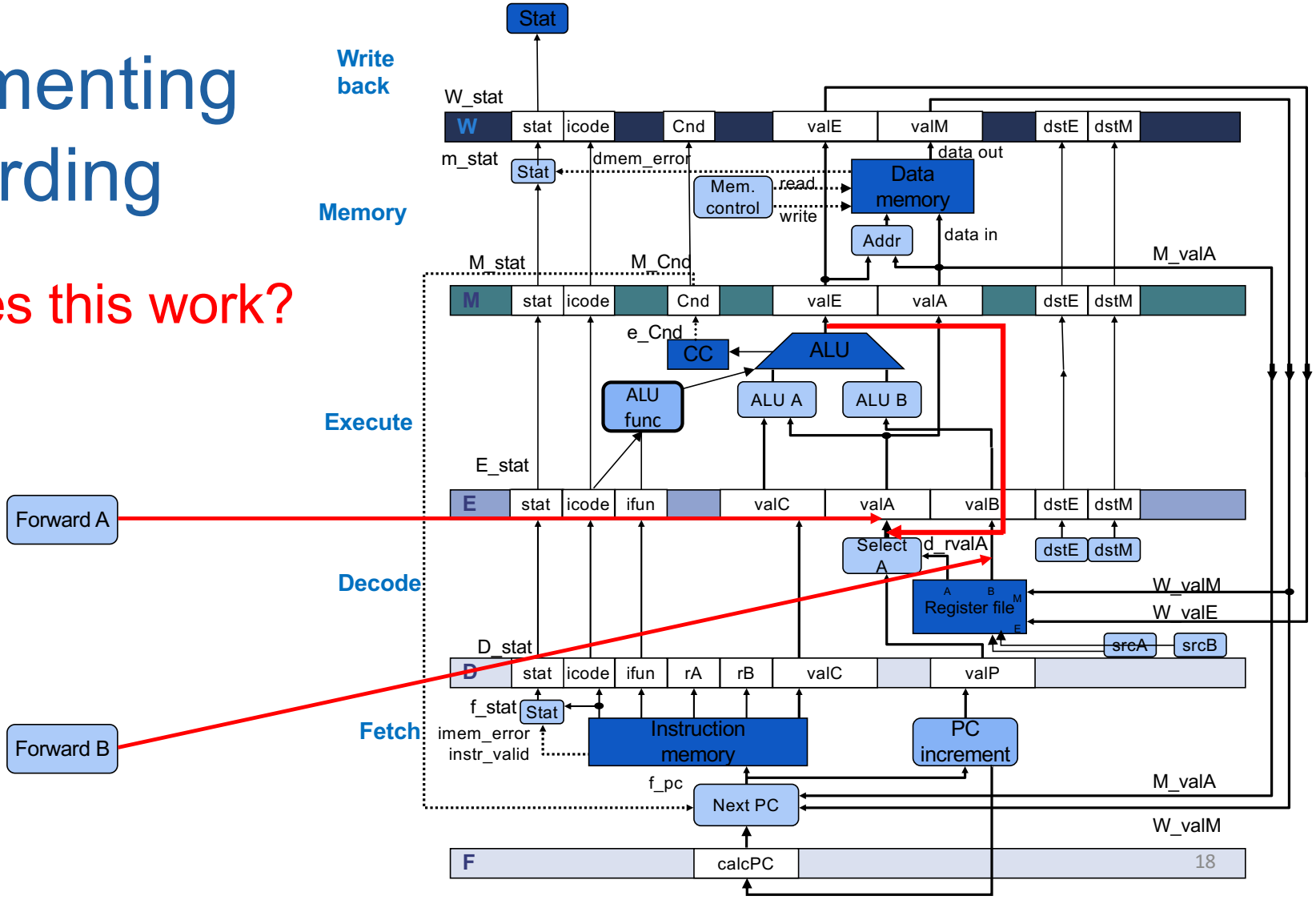


Implementing Forwarding



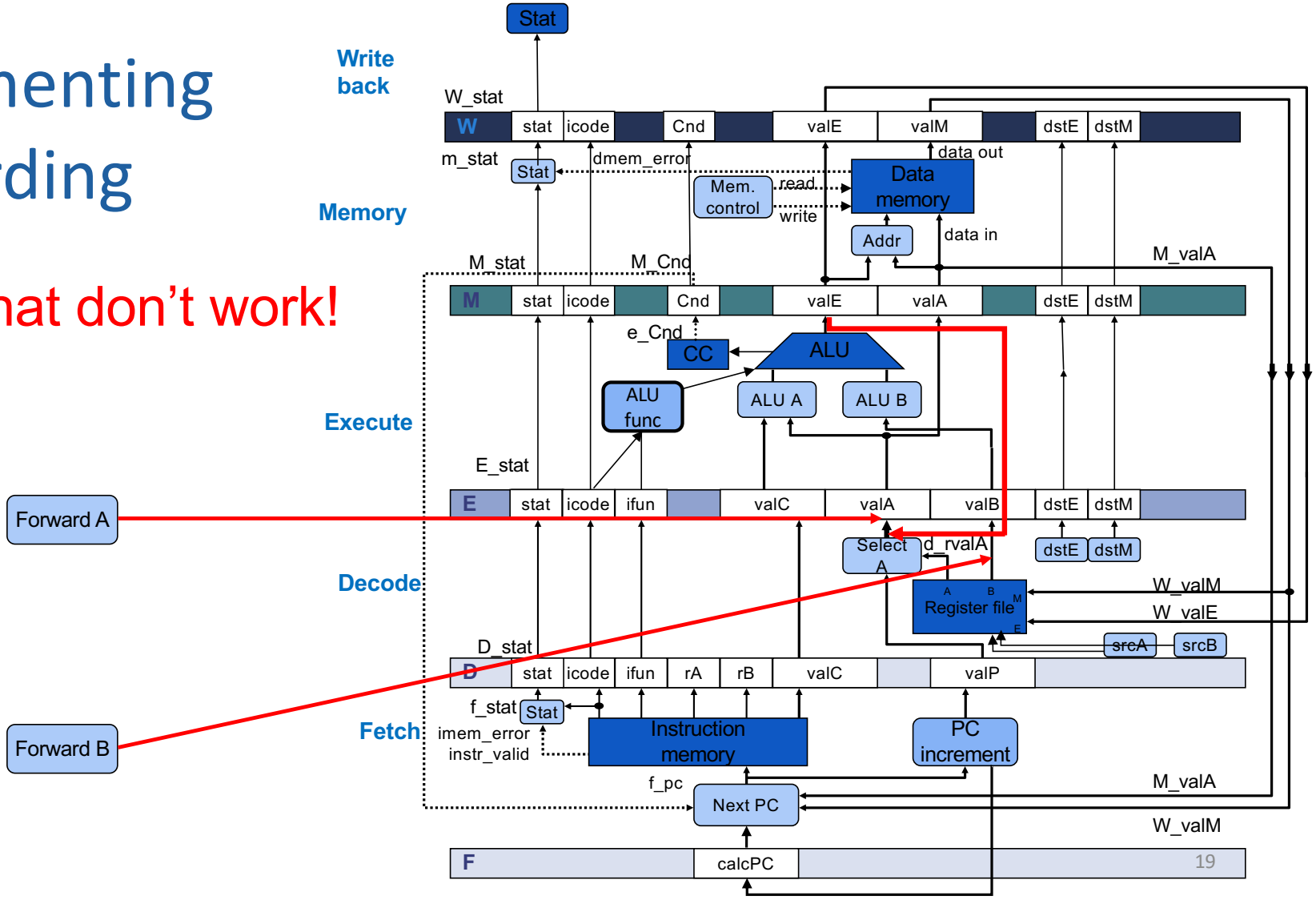
Implementing Forwarding

Why does this work?



Implementing Forwarding

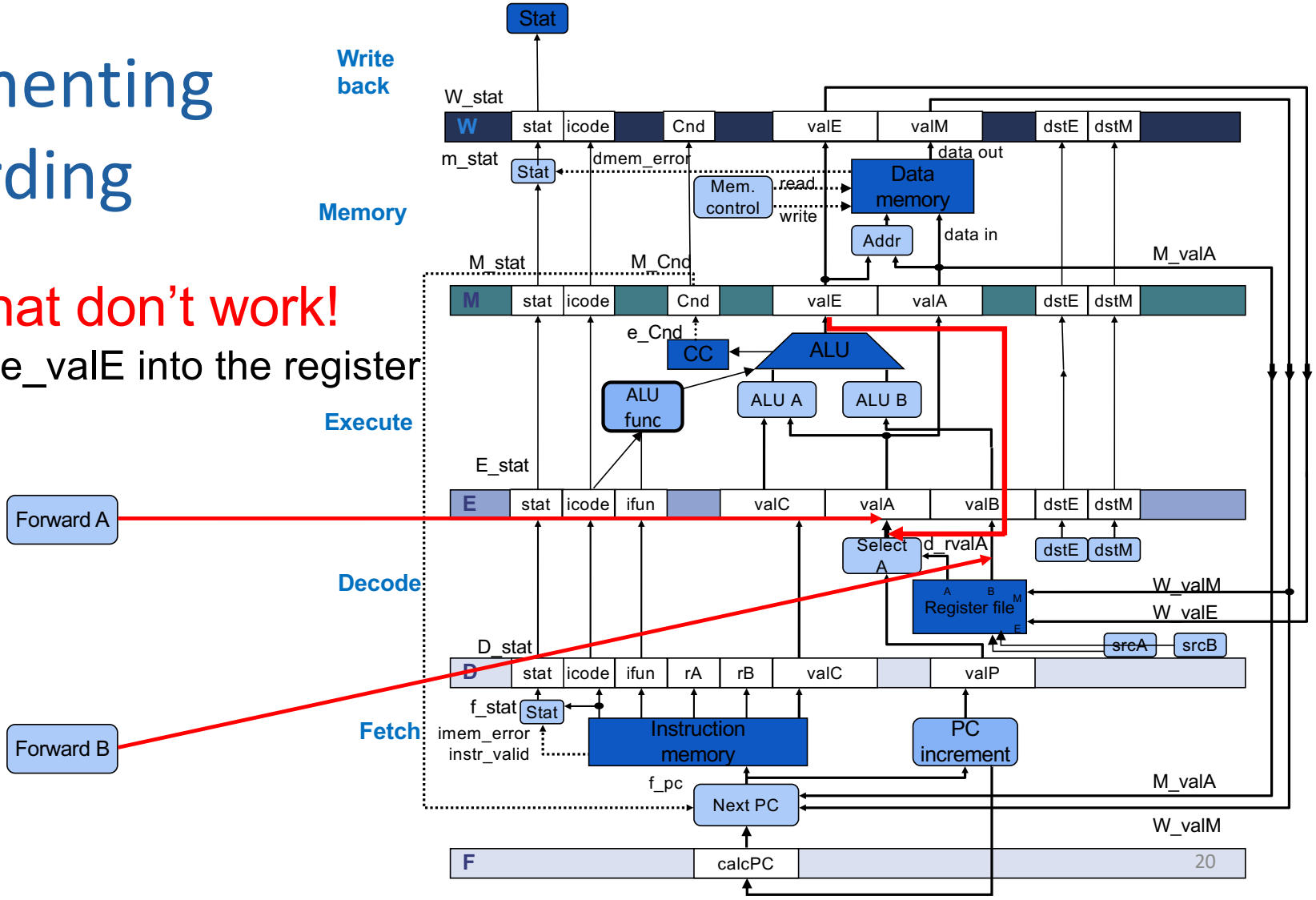
Things that don't work!



Implementing Forwarding

Things that don't work!

1. forward e_valE into the register file



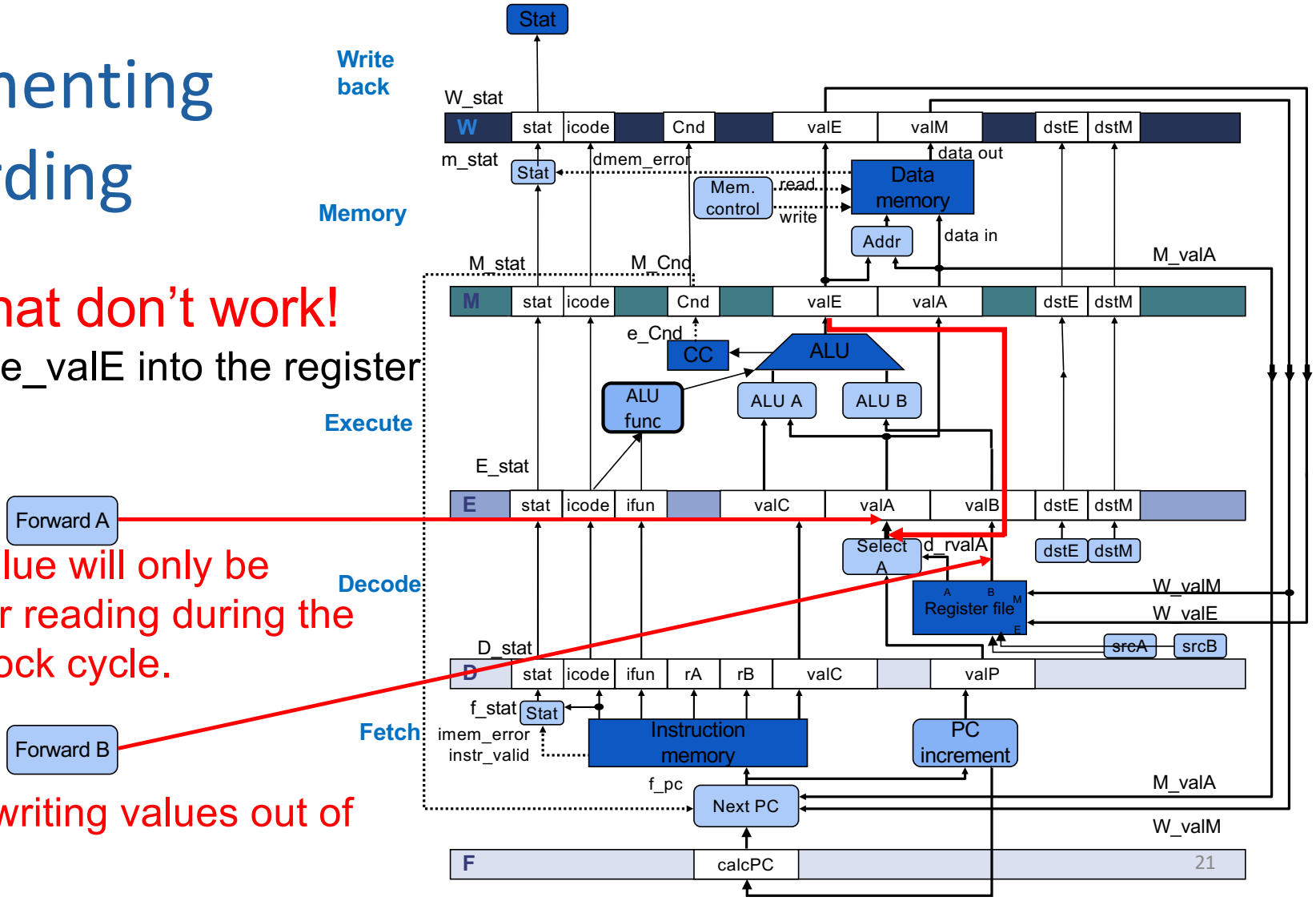
Implementing Forwarding

Things that don't work!

1. forward e_valE into the register file

The new value will only be available for reading during the following clock cycle.

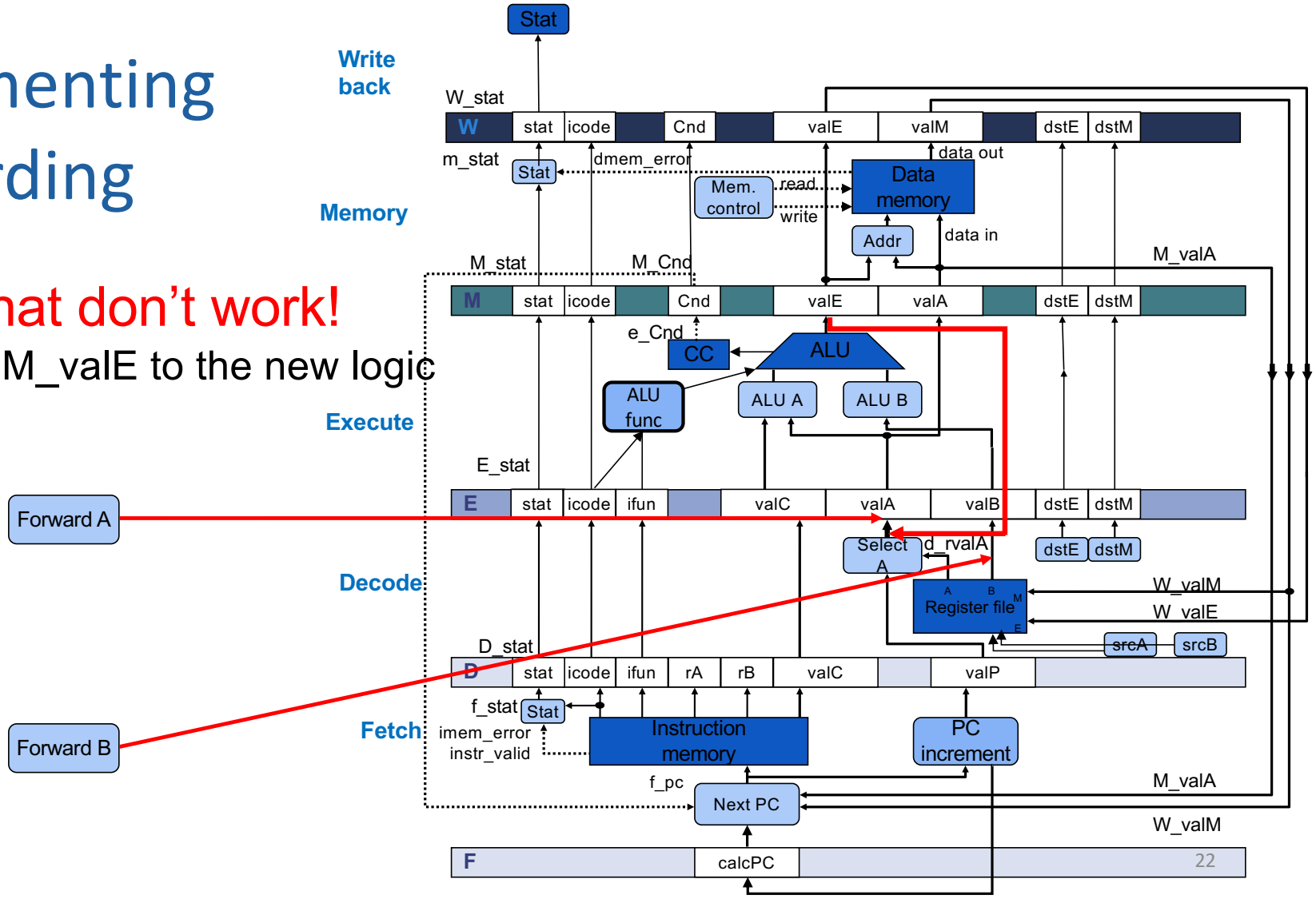
Also we're writing values out of order.



Implementing Forwarding

Things that don't work!

2. forward M_valE to the new logic

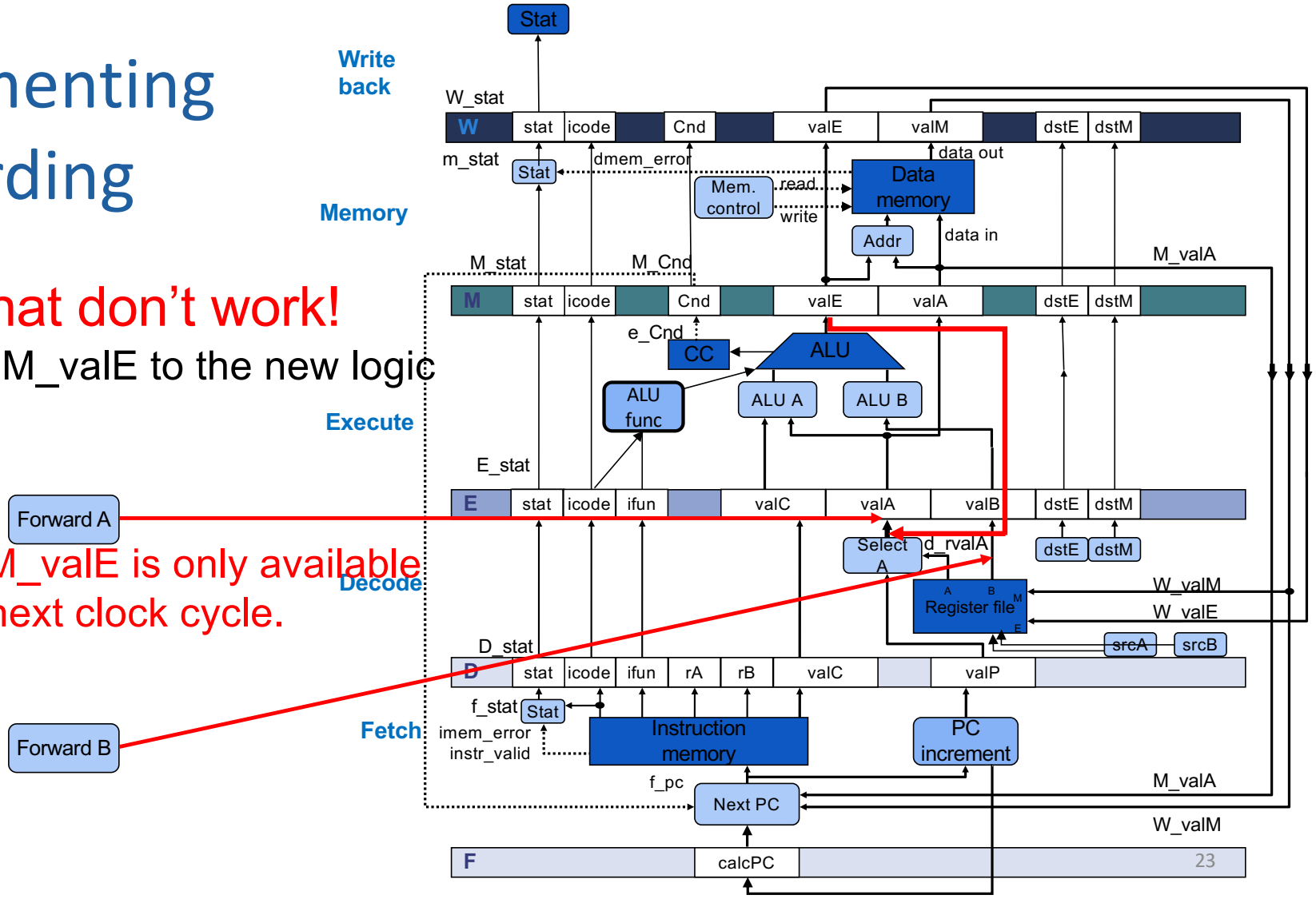


Implementing Forwarding

Things that don't work!

2. forward M_valE to the new logic

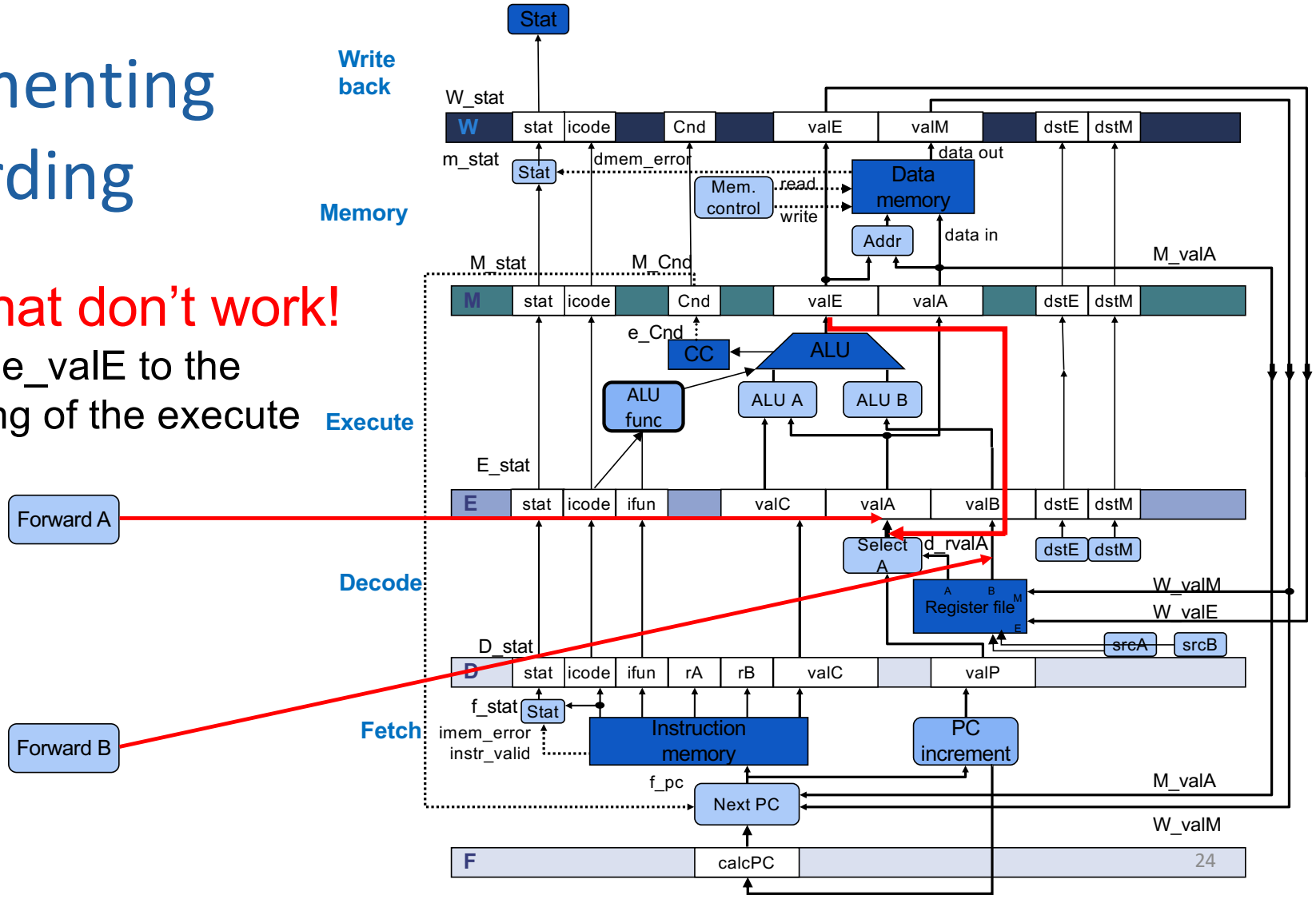
The value M_valE is only available during the next clock cycle.



Implementing Forwarding

Things that don't work!

3. forward e_valE to the beginning of the execute stage

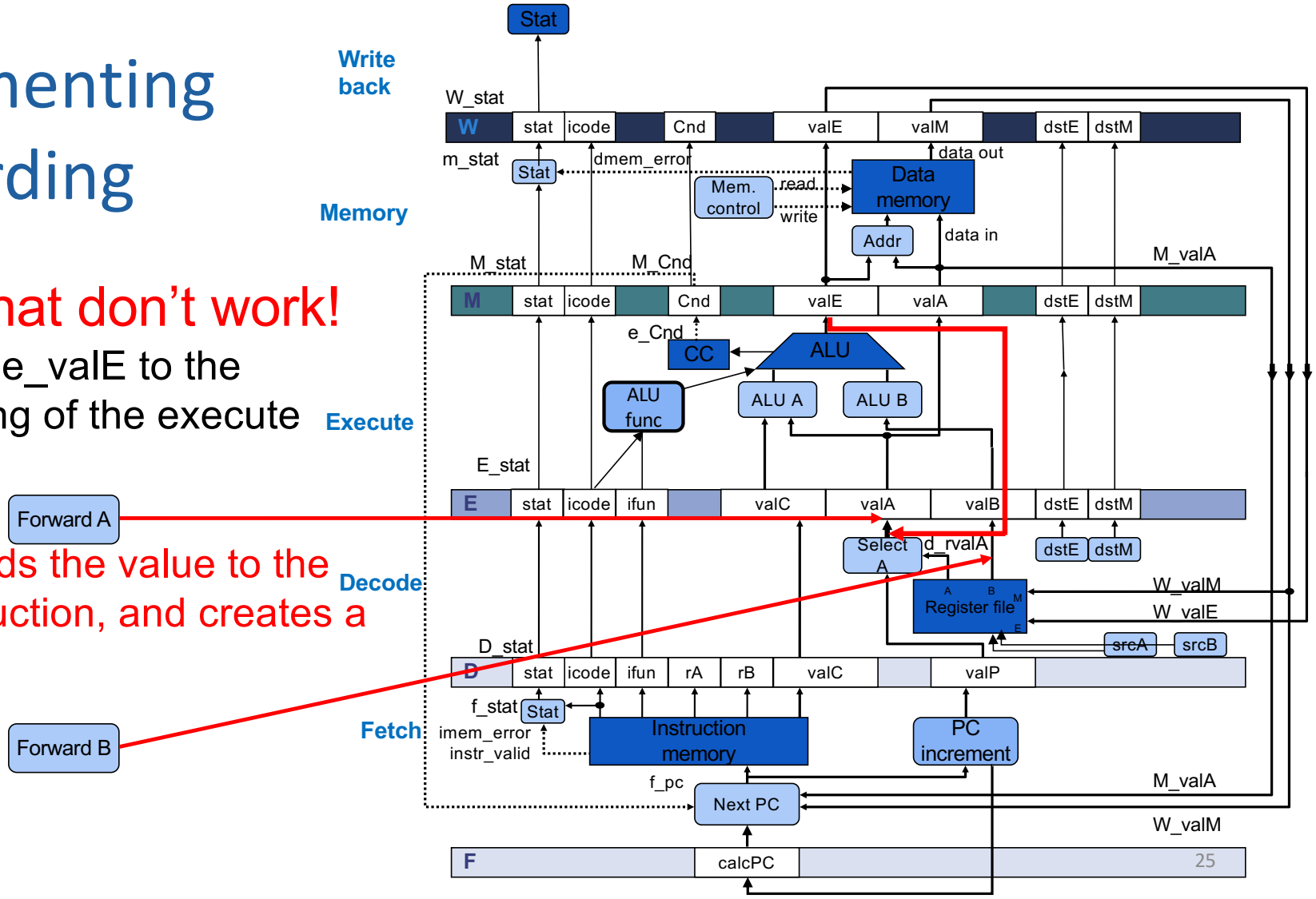


Implementing Forwarding

Things that don't work!

3. forward e_valE to the beginning of the execute stage

This forwards the value to the **same** instruction, and creates a loop.

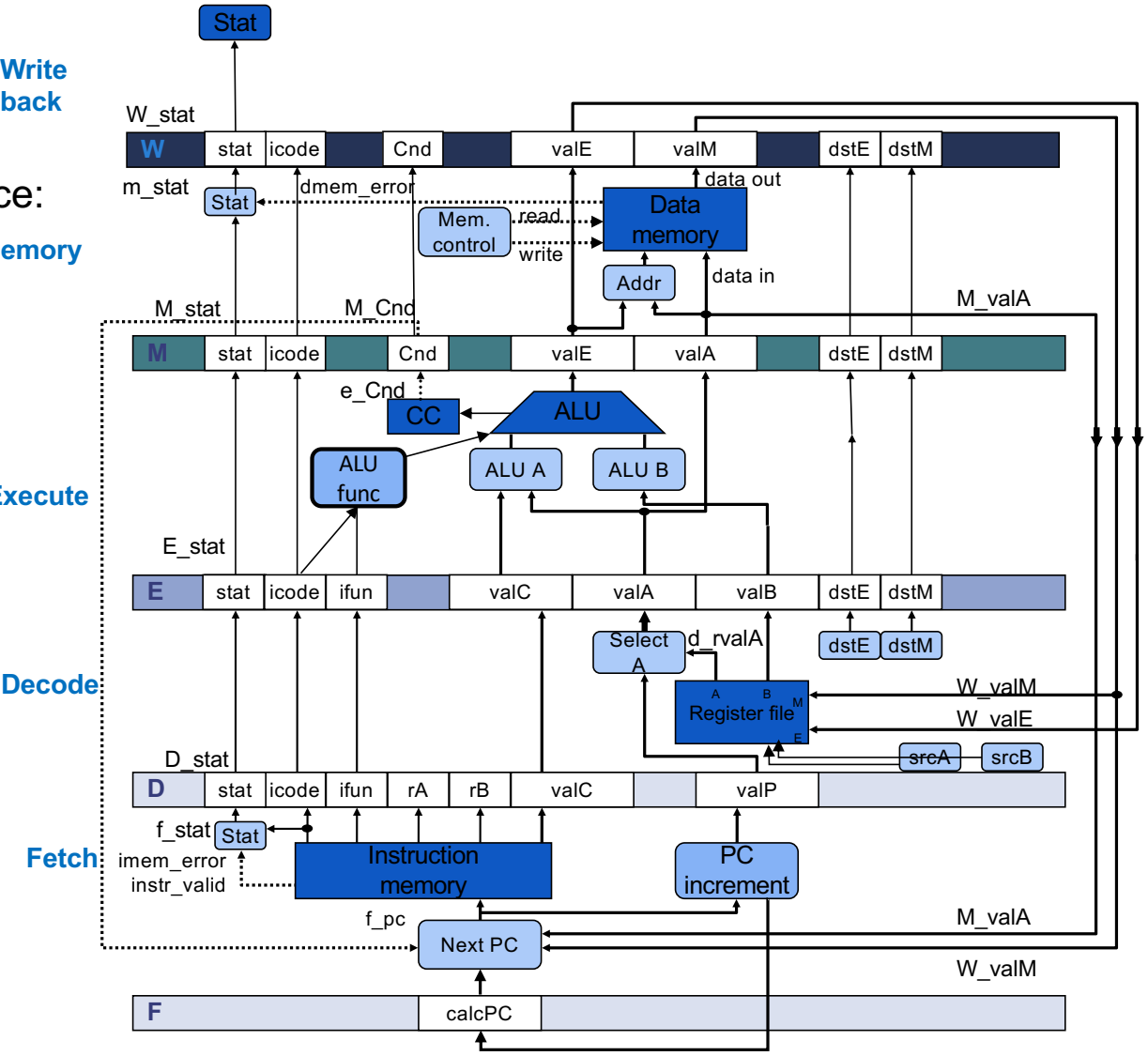


Hey, Quick Q..

Consider now the following sequence:

addq %rax, %rbx
irmovq 10, %rdx
subq %rbx, %rcx

NOW at what point does subq need the value of %rbx? Same spot?

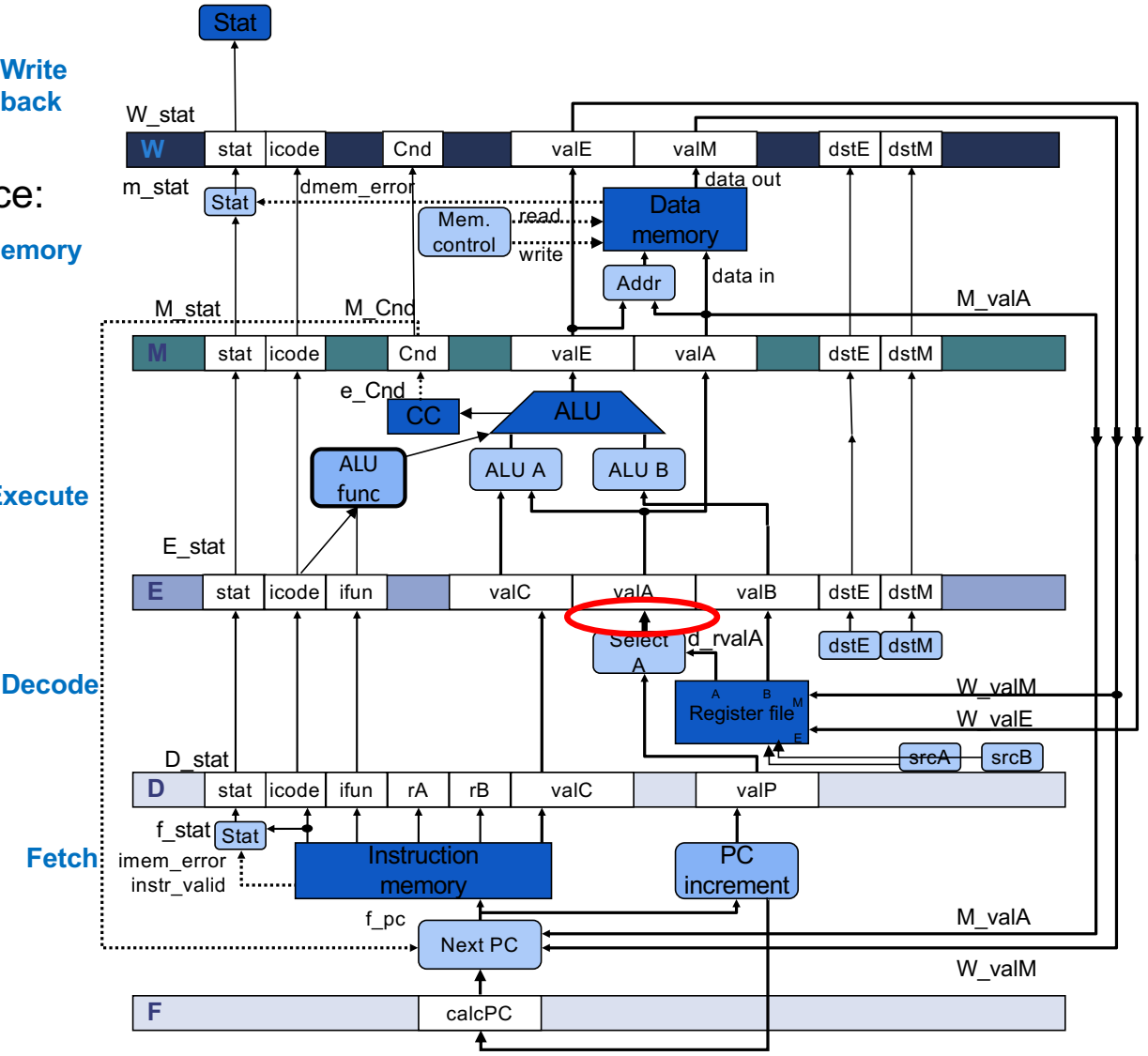


Hey, Quick Q..

Consider now the following sequence:

addq %rax, %rbx
irmovq 10, %rdx
subq %rbx, %rcx

NOW at what point does subq need the value of %rbx? Same spot?



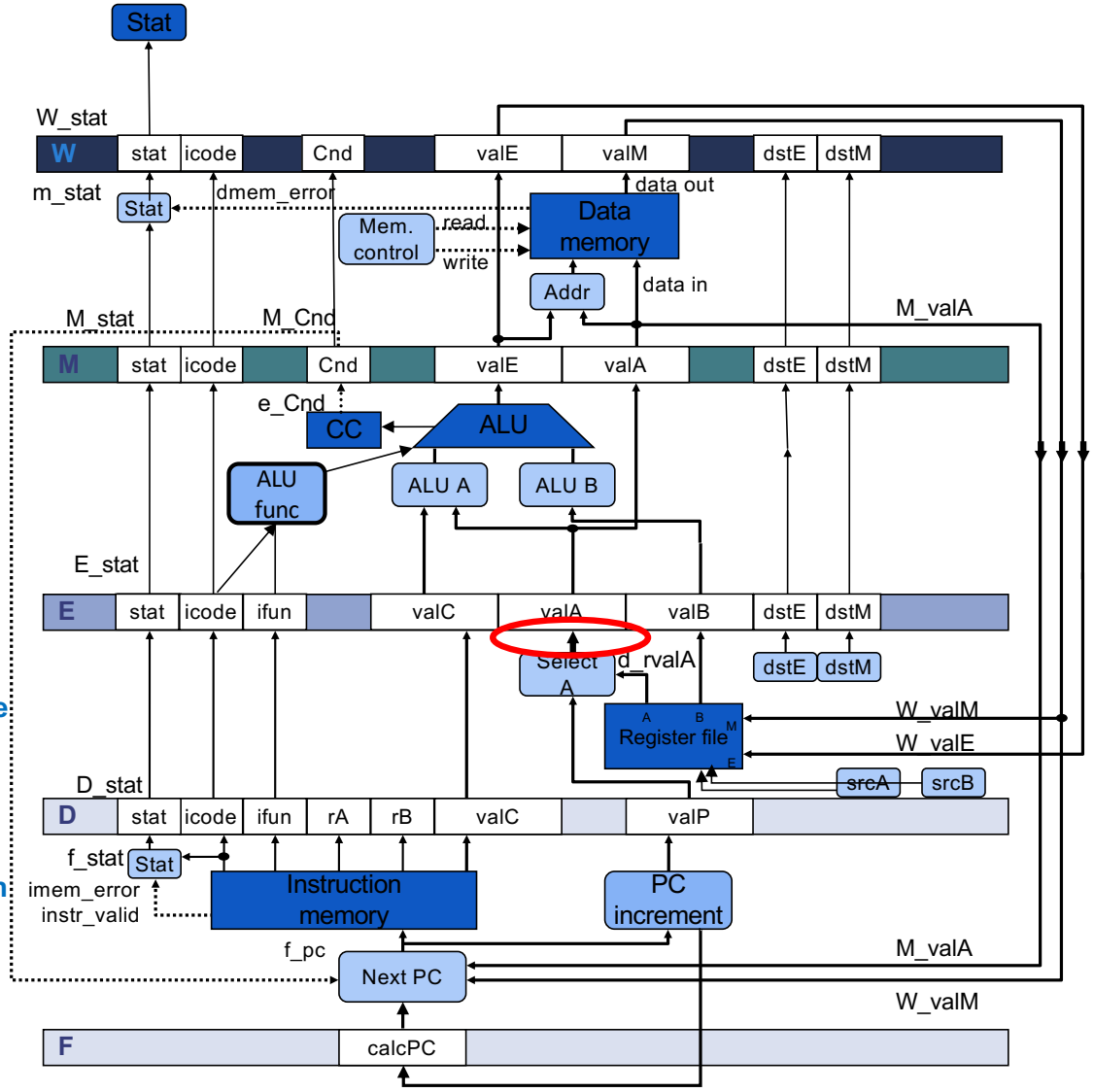
Hey, Quick Q..

Consider now the following sequence:

addq %rax, %rbx
irmovq 10, %rdx
subq %rbx, %rcx

NOW at what point does subq need the value of %rbx? Same spot?

Exactly where is the value of %rbx when it's needed? Same spot?



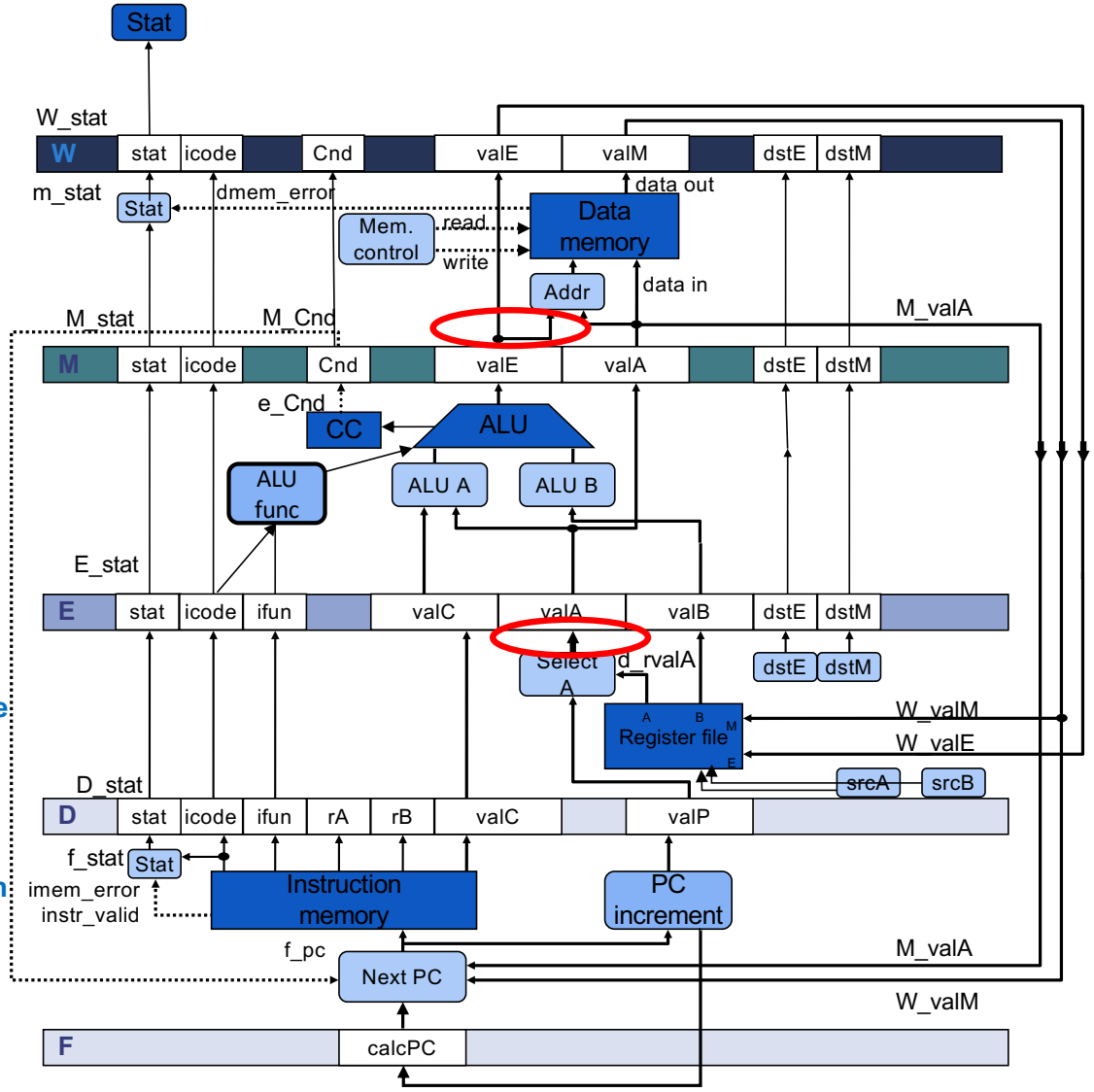
Hey, Quick Q..

Consider now the following sequence:

addq %rax, %rbx
irmovq 10, %rdx
subq %rbx, %rcx

NOW at what point does subq need the value of %rbx? Same spot?

Exactly where is the value of %rbx when it's needed? Same spot?



How does the processor keep track of all this?

- In some sense, just like before: with logic blocks!
- Logic that reasons about instructions at multiple stages and reasons about their interactions: messier, but not really different.
- You can read about pipeline control logic in the textbook.
- For CPSC 313: you should a) know that such logic exists and b) conceptually know what it has to do and c) be able to talk about how it could be extended/changed as needed.

In-class Activity

- Explore other forwarding options

- You can repeat the questions until you feel comfortable

Wrapping Up

- Forwarding lets us mitigate some of the hazards that occur due to pipelining.
- In early RISC processors, there was no stalling and no forwarding (chips could not contain as many transistors and wires).
  - The compiler was responsible for inserting nops into the assembly code to ensure correct execution!
  - When humans had to write assembly directly (which occurs in some of the lowest levels of the operating system), those programmers had to be acutely aware of the internals of the hardware.
- Today, the hardware is so complex that it has to take care of hazards, not people or compilers.